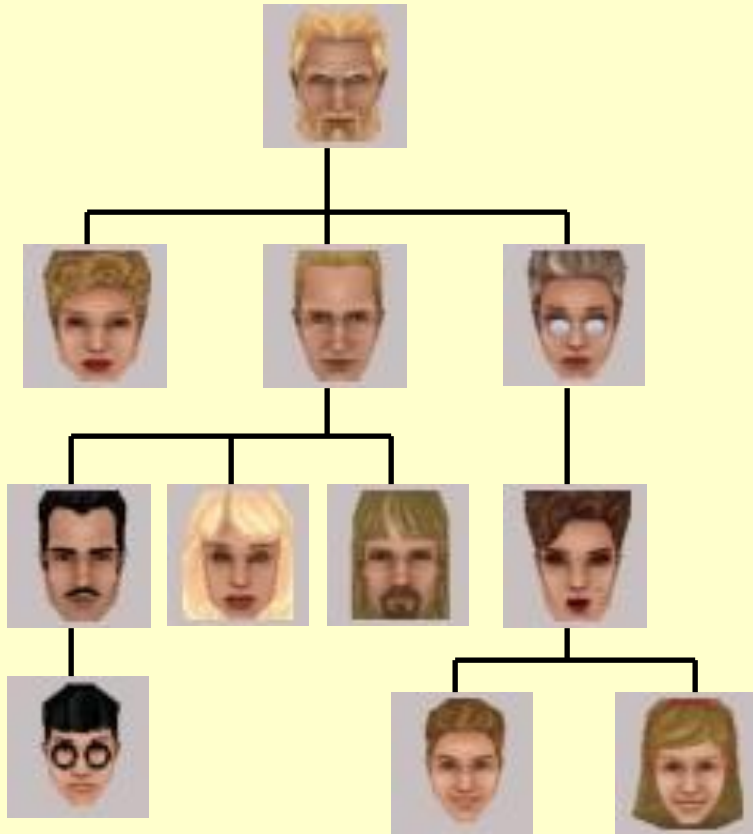


第6章

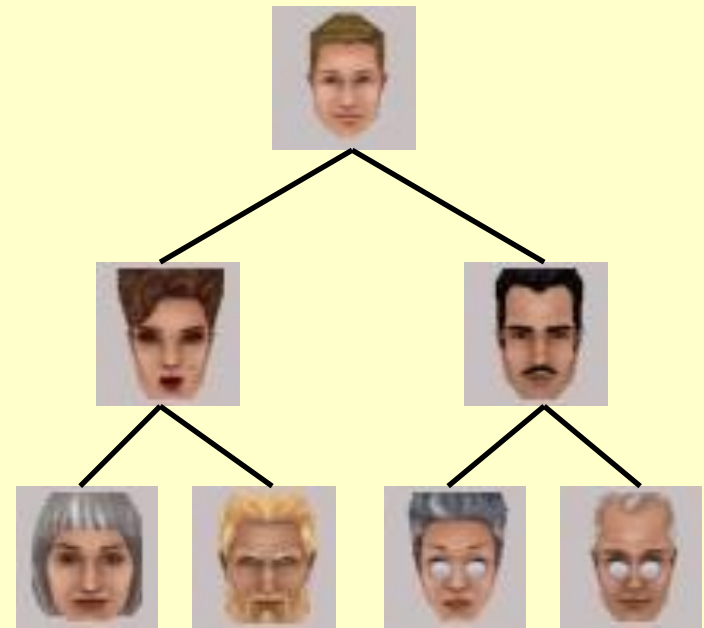
§ 6.1 基本知识

TREES树和二叉树

1. Terminology术语



Lineal Tree家谱



Pedigree Tree谱系树
(binary tree)

【Definition】 定义: A **tree** is a collection of nodes. The collection can be empty; otherwise, a tree consists of

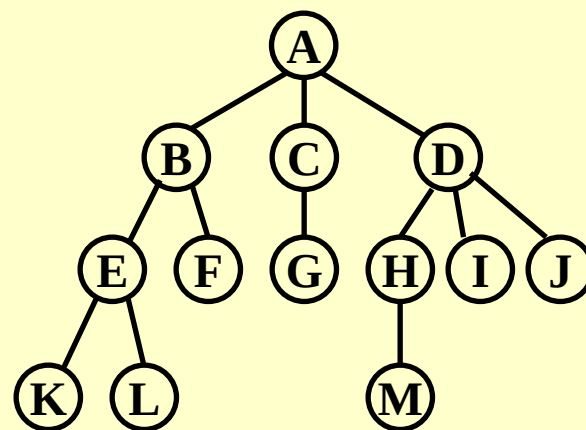
- (1) 唯一的一个根结点 a distinguished node **r** , called the **root**;
- (2) 零个或多个子树 and zero or more nonempty **(sub)trees** 子树 $T_1, \dots, T_k$, each of whose roots are connected by a directed **edge** from r .

Note:

- 子树间不连接 Subtrees must not connect together.
Therefore every node in the tree is the root of some subtree.
- There are **$N - 1$** edges in a tree with N nodes.
- Normally the root is drawn at the top. 从上向下画

✎ **degree of a node** 结点的度 ::= 子树个数
number of subtrees of the node. For
example, **degree(A) = 3, degree(F) = 0.**

✎ **degree of a tree** ::= $\max_{\text{node} \in \text{tree}} \{ \text{degree}(\text{node}) \}$
树的度 For example, **degree of this tree = 3.**



✎ **parent** ::= a node that has subtrees.
双亲节点或父节点

Parent of I: ?
Parent of L: ?

✎ **children** ::= the roots of the subtrees of a parent.
孩子结点或子结点

Children of A: ?
Children of B: ?

✎ **siblings** ::= children of the same parent.
兄弟结点

B, C, D
K, L

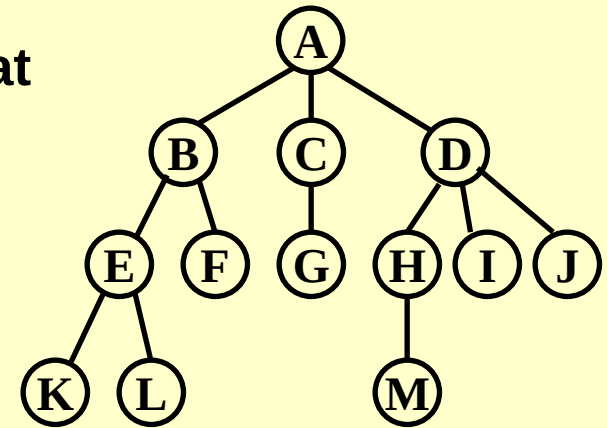
✎ **leaf (terminal node)** ::= a node with degree 0 (no children).

叶子结点

leaf: K, L, F, G, M, I, J

✎ 路径 **path from n_1 to n_k ::= a (unique)**
sequence of nodes $n_1, n_2, \dots, n_k$ such that
 n_i is the parent of n_{i+1} for $1 \leq i < k$.

✎ 路径长度 **length of path ::= number of**
edges on the path.



✎ **height (depth) of a tree ::= 树高度或深度**, 树中结点的最大层次,
如例为4

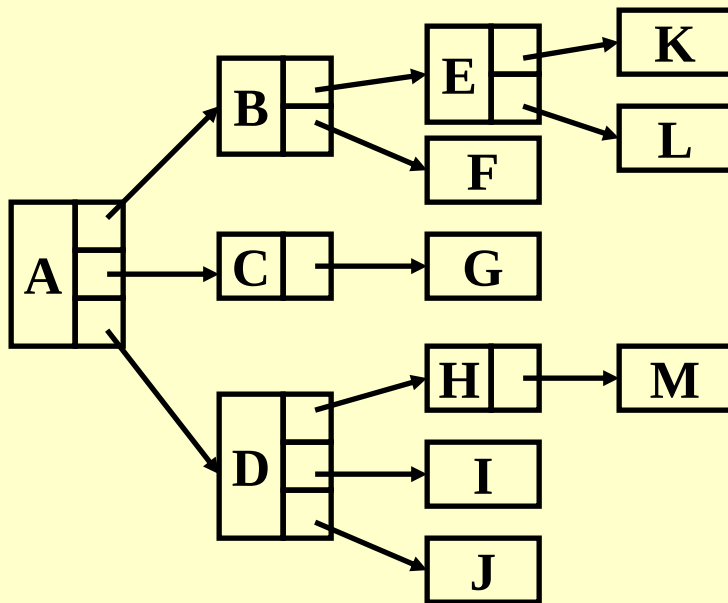
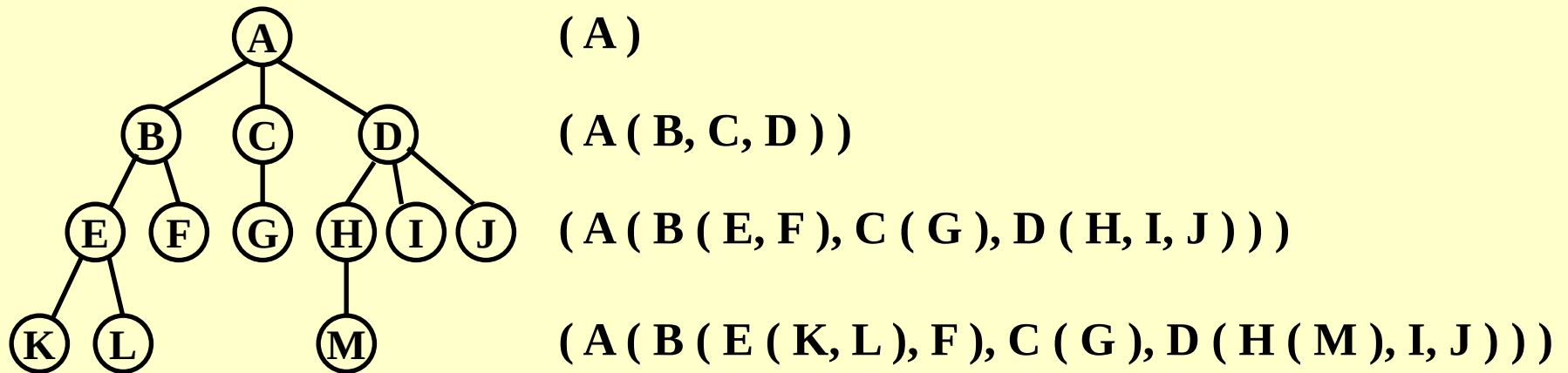
✎ 祖先结点 **ancestors of a node ::= all the nodes along the path**
from the node up to the root. L:?

✎ 后代结点 **descendants of a node ::= all the nodes in its**
subtrees.

B:?

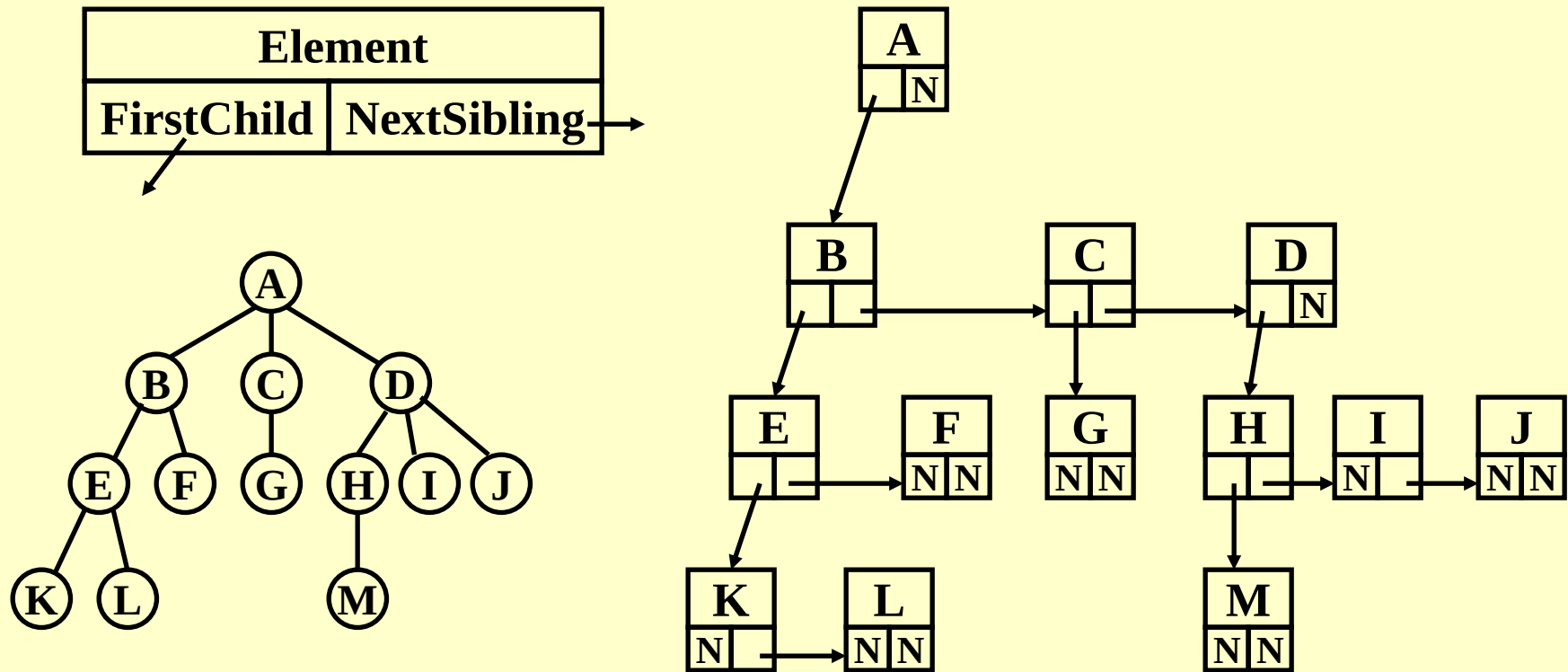
2. 实现Implementation

❖ List Representation链表表示



So the size of each node depends on the number of branches.
Hmmm... That's not good.

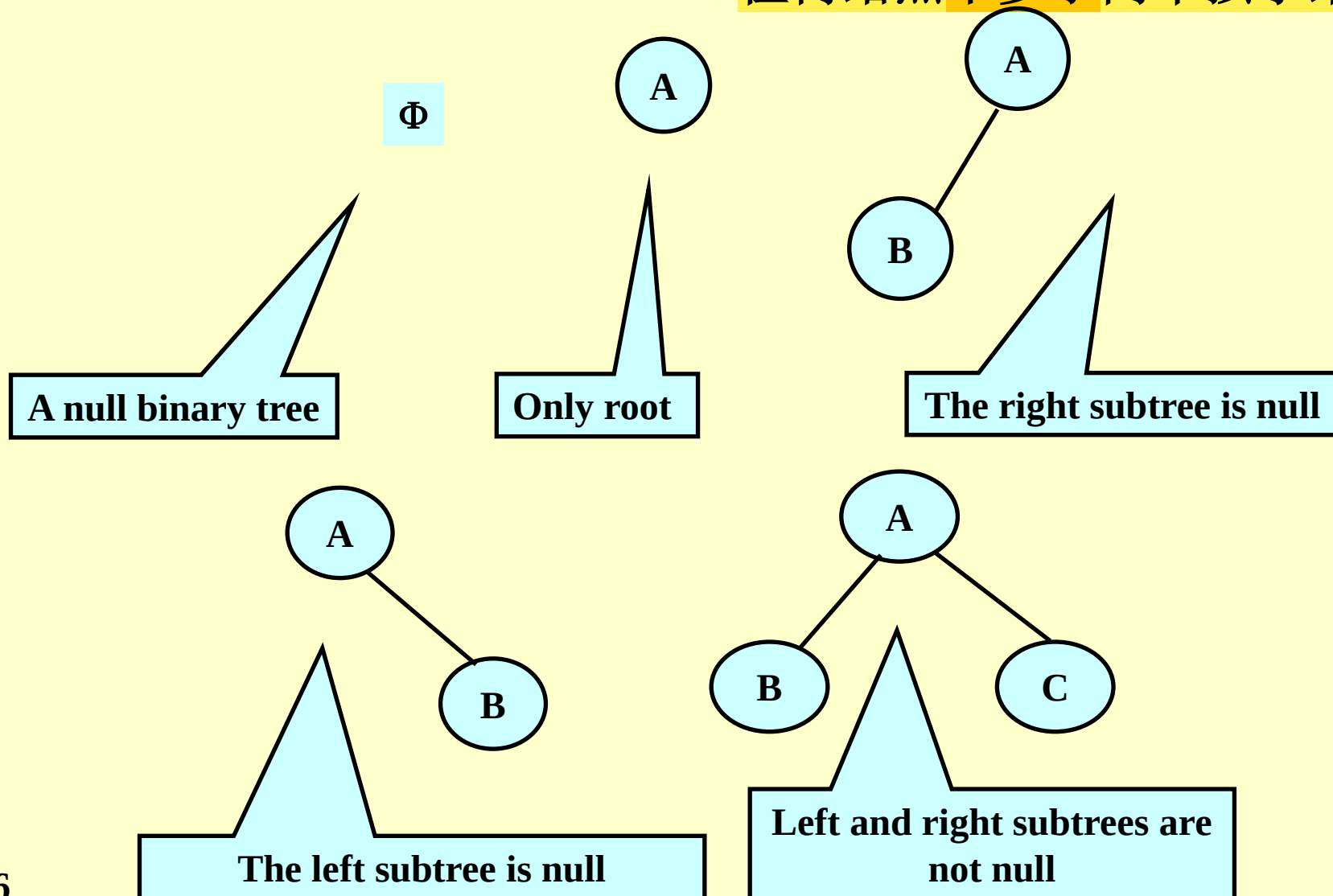
❖ FirstChild-NextSibling Representation



Note: The representation is **not unique** since the children in a tree can be of any order.

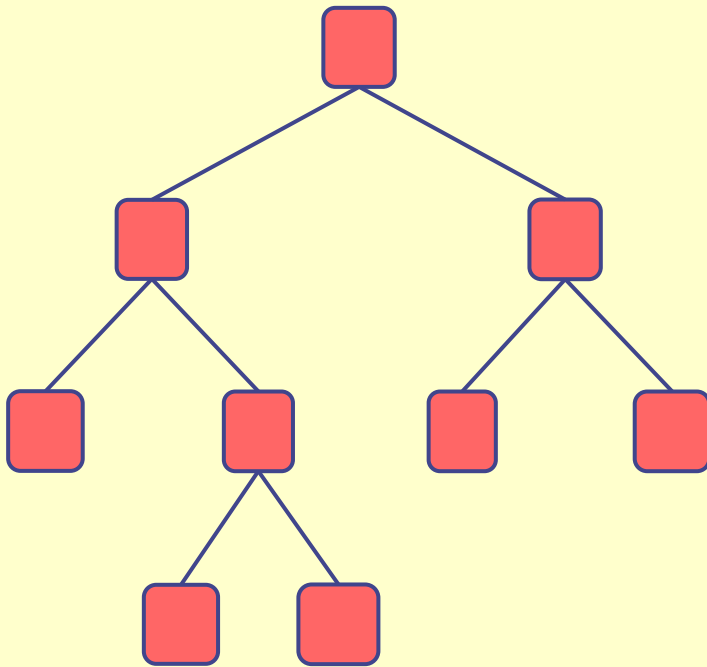
§ 6.2 二叉树 Binary Trees

【Definition】 A **binary tree** is a tree in which no node can have more than two children. 任何结点不多于两个孩子结点

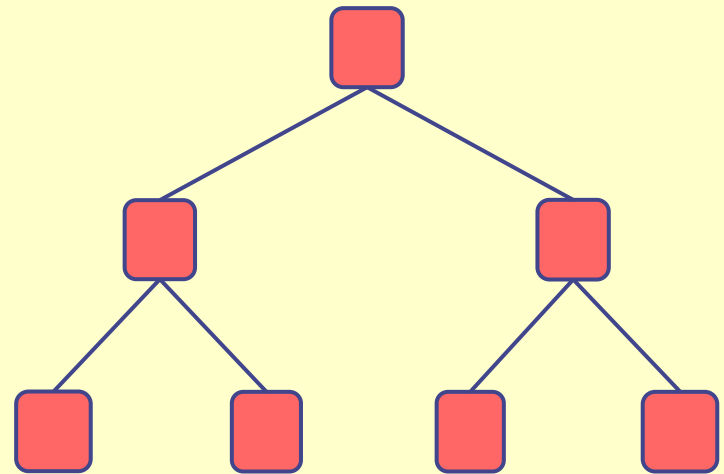


Perfection(满二叉树)

- A binary tree is a **perfect binary tree** if every level is completely full 每层全满



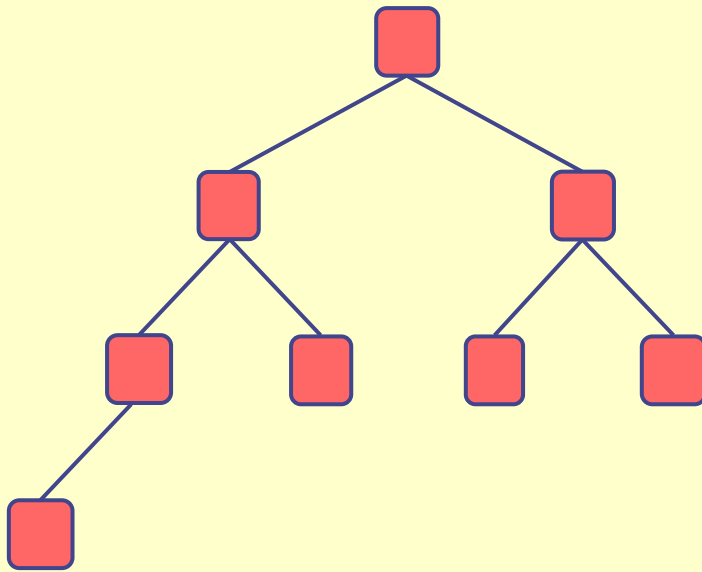
Not perfect



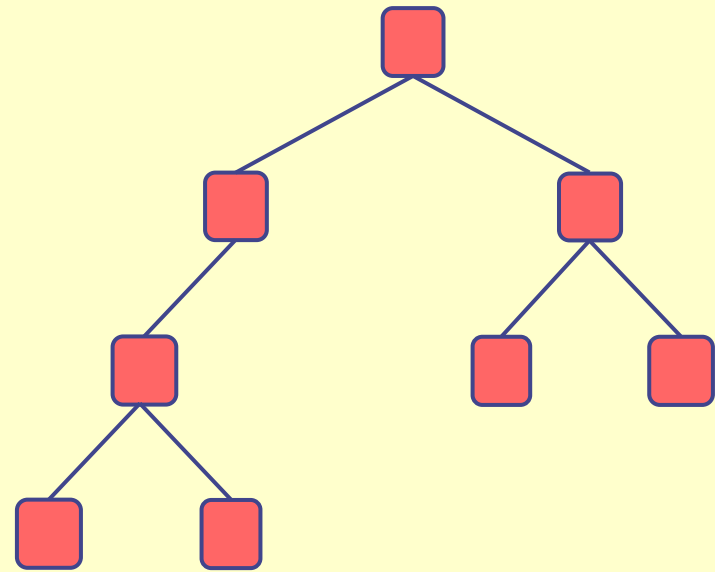
满二叉树Perfect!

Completeness(完全二叉树)

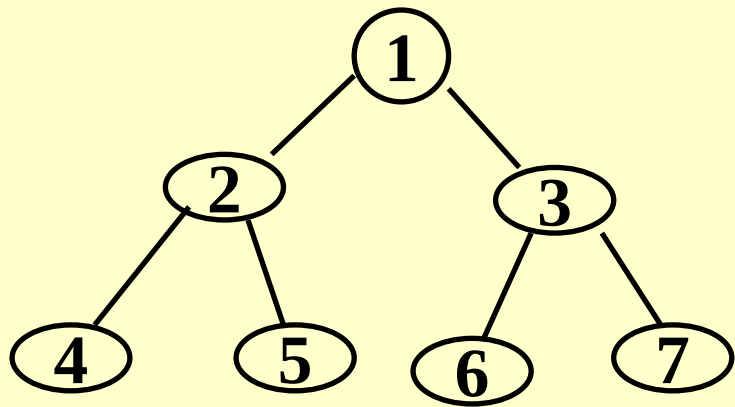
- A binary tree is a **left-complete binary tree** if:
 - every level is completely full, possibly excluding the lowest level 除最后一层结点全满
 - all nodes are as far left as possible 所有结点尽可能靠左



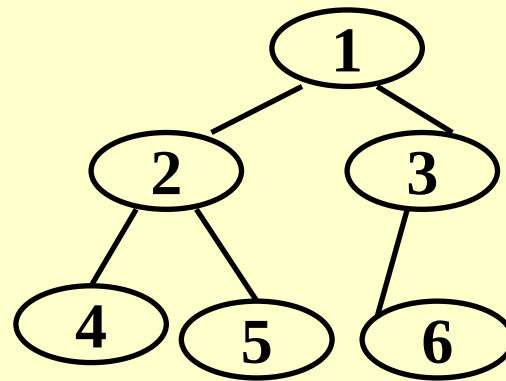
Left-complete!



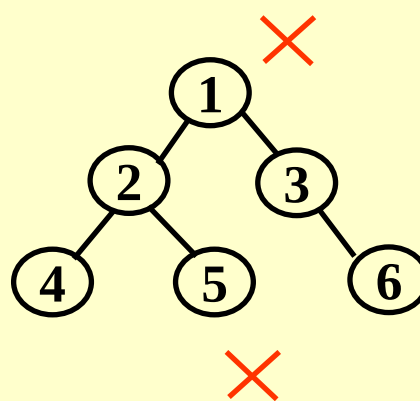
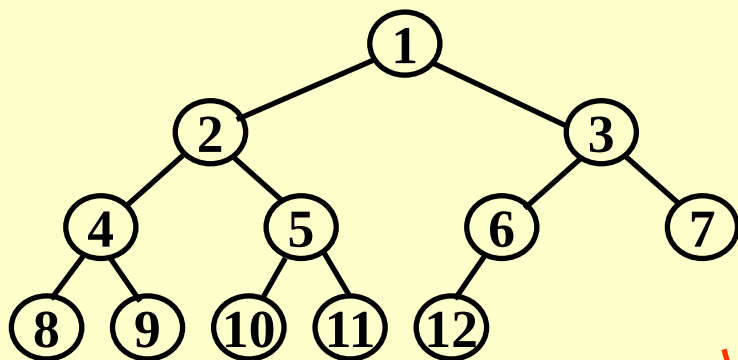
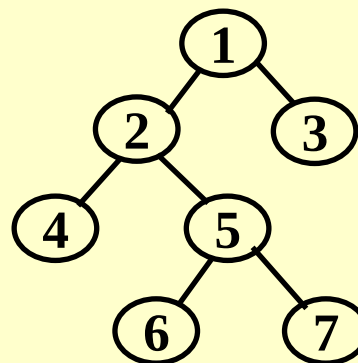
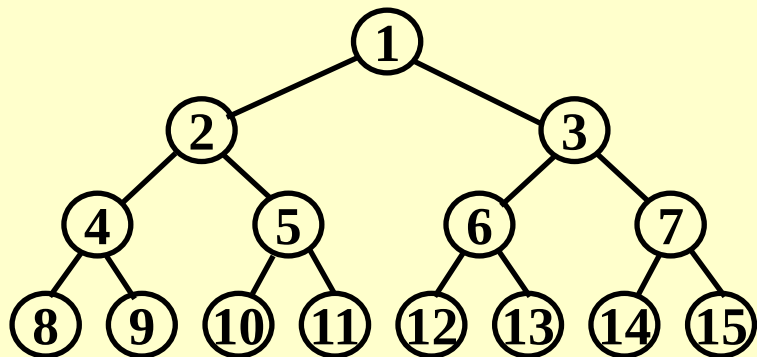
Not left-complete



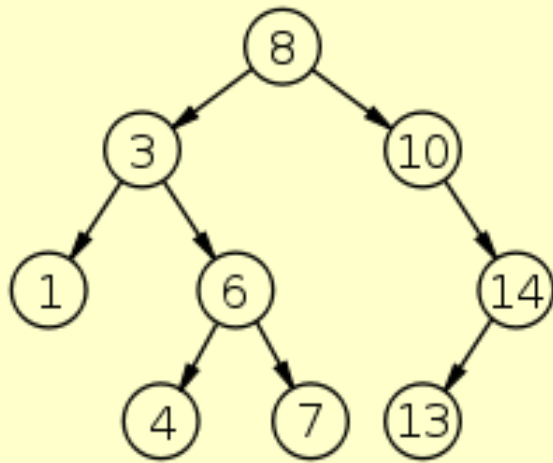
满二叉树 A full binary tree



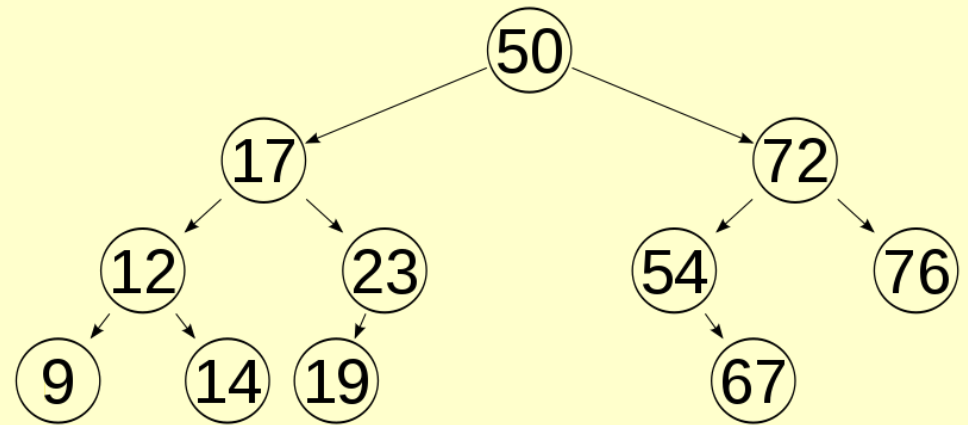
完全二叉树 A complete binary tree



?

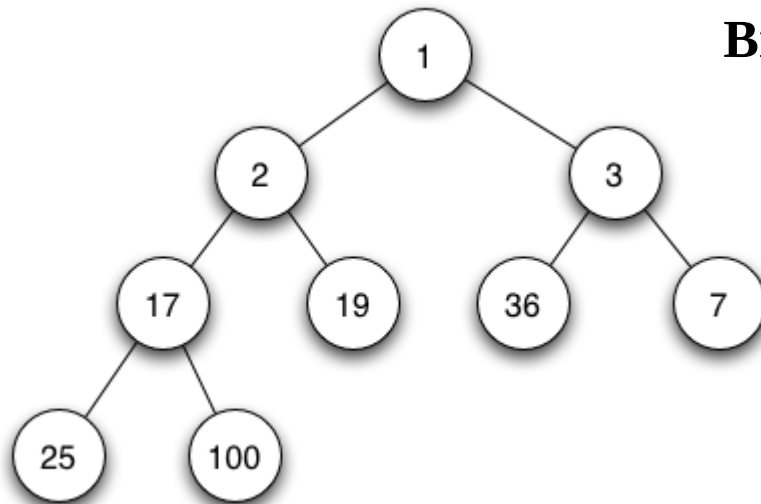


BST: a binary search tree
 二叉搜索树

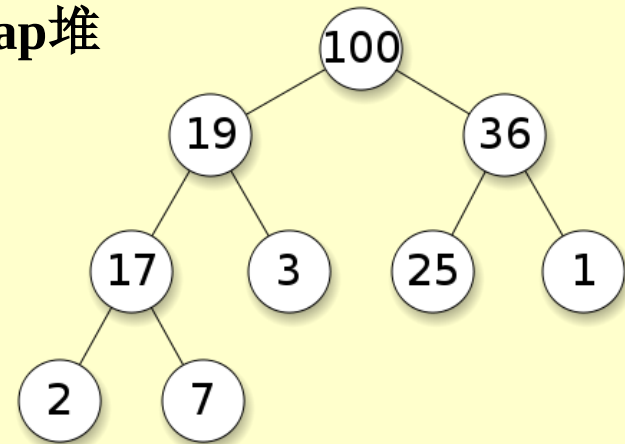


AVL tree: a self-balancing binary search tree
 自平衡的

Binary Heap堆



a complete binary min heap



a complete binary max heap

二叉树的性质

- 性质1：在二叉树的第 i 层上至多有 2^{i-1} 个结点($i \geq 1$);
- 性质2：深度为 k 的二叉树至多有 $2^k - 1$ 个结点($k \geq 1$);
- 性质3：对任何一棵二叉树 T ，如果其叶子结点数为 n_0 ，度为2的结点数为 n_2 ，则 $n_0 = n_2 + 1$
- 性质4：具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$
- 性质5：如果对一棵有 n 个结点的完全二叉树的结点按层序编号(从第1层到第 $\lfloor \log_2 n \rfloor + 1$ 层，每层从左到右),则对任一结点 i ($1 \leq i \leq n$),则:
 - (1) 如果 $i = 1$ ，则结点 i 无双亲，是二叉树的根；如果 $i > 1$ ，则其双亲是结点 $\lfloor i/2 \rfloor$ 。
 - (2) 如果 $2i > n$ ，则结点 i 为叶子结点，无左孩子；否则($2i \leq n$)，其左孩子是结点 $2i$ 。
 - (3) 如果 $2i + 1 > n$ ，则结点 i 无右孩子；否则($2i + 1 \leq n$)，其右孩子是结点 $2i + 1$ 。

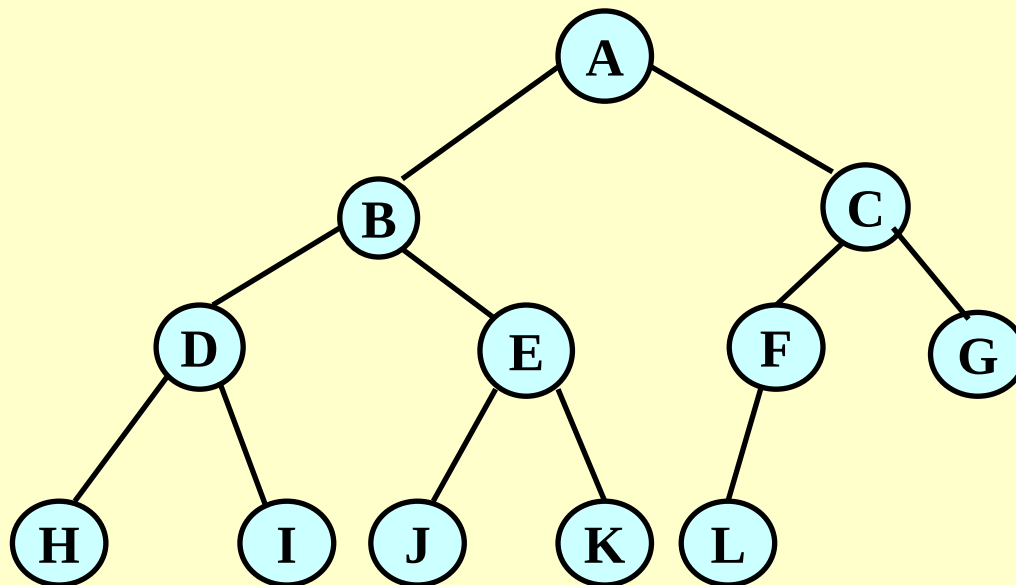
❖ 顺序结构实现

```
#define max_tree_size 100
```

```
typedef telement sqbitree[max_tree_size];
```

```
Sqbitree bt;
```

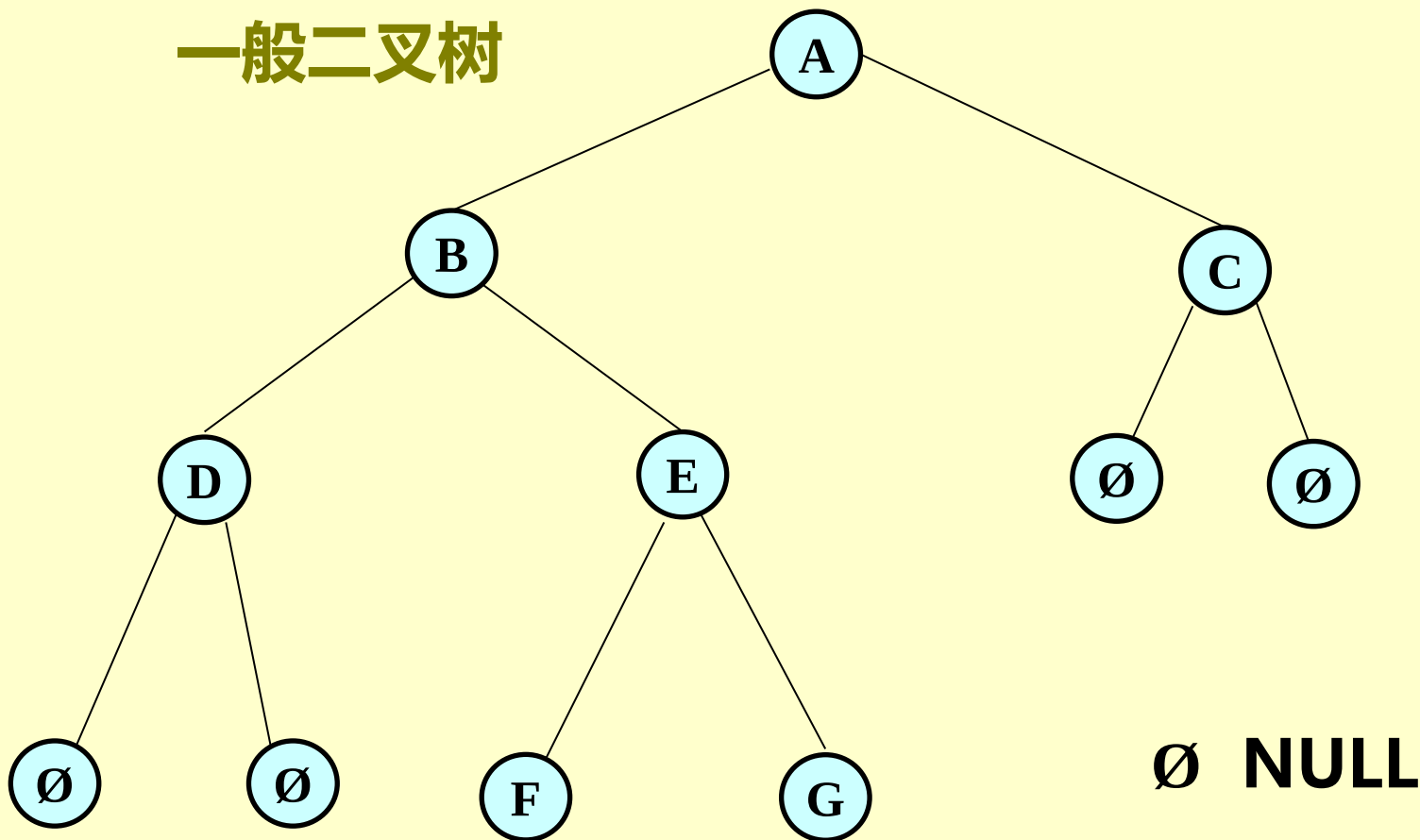
完全二叉树



1 2 3 4 5 6 7 8 9 10 11 12

A	B	C	D	E	F	G	H	I	J	K	L
---	---	---	---	---	---	---	---	---	---	---	---

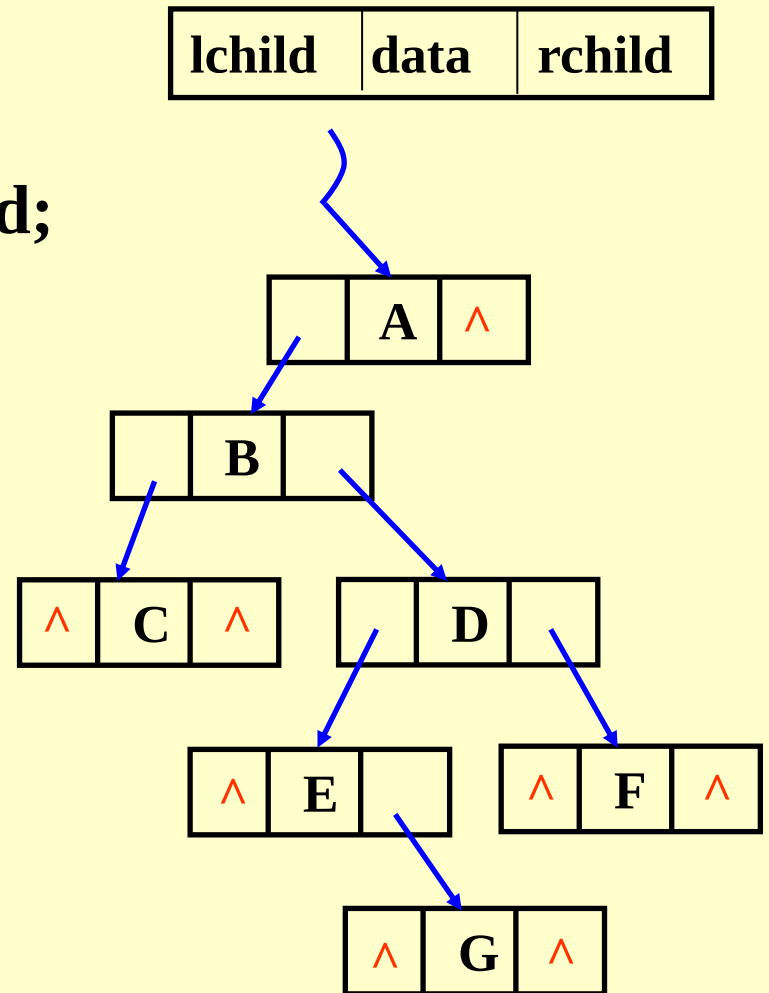
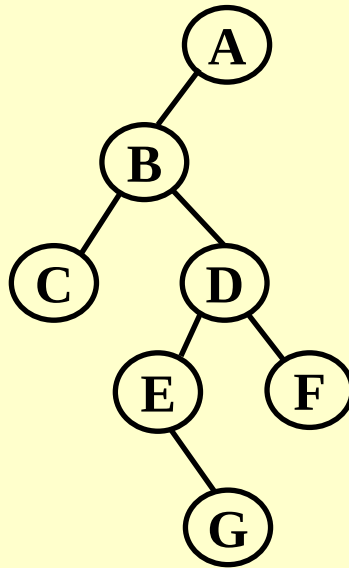
一般二叉树



A	B	C	D	E	∅	∅	∅	∅	F	G
---	---	---	---	---	---	---	---	---	---	---

❖ 二叉链表实现

```
typedef struct BiTNode {  
    TelemType data;  
    struct BiTNode *lchild, *rchild;  
} BiTNode, *BiTree;
```



n个结点的二叉树有n+1个空指针

❖ 二叉树创建

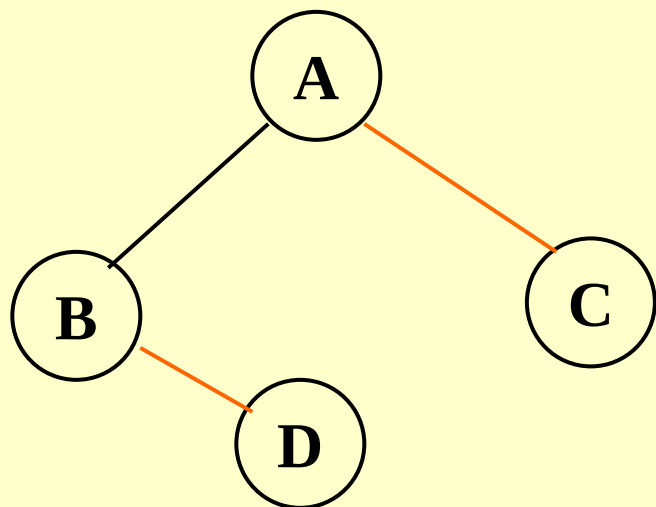
// 创建二叉树，输入#创建空结点.

```
BiTree CreateBiTree(BiTree T){  
    char ch;  
    scanf(" %c", &ch);  
    if(ch=='#')  
        T=NULL;  
    else{  
        if(!(T=(BiTNode *)malloc(sizeof(BiTNode))))  
            exit(OVERFLOW);  
        T->data=ch;           // to create the root node  
        T->lchild=CreateBiTree(T->lchild);  // 递归创建左子树  
        T->rchild=CreateBiTree(T->rchild);  //递归创建右子树  
    }  
    return T;  
}
```


❖ 二叉树遍历 访问结点一次

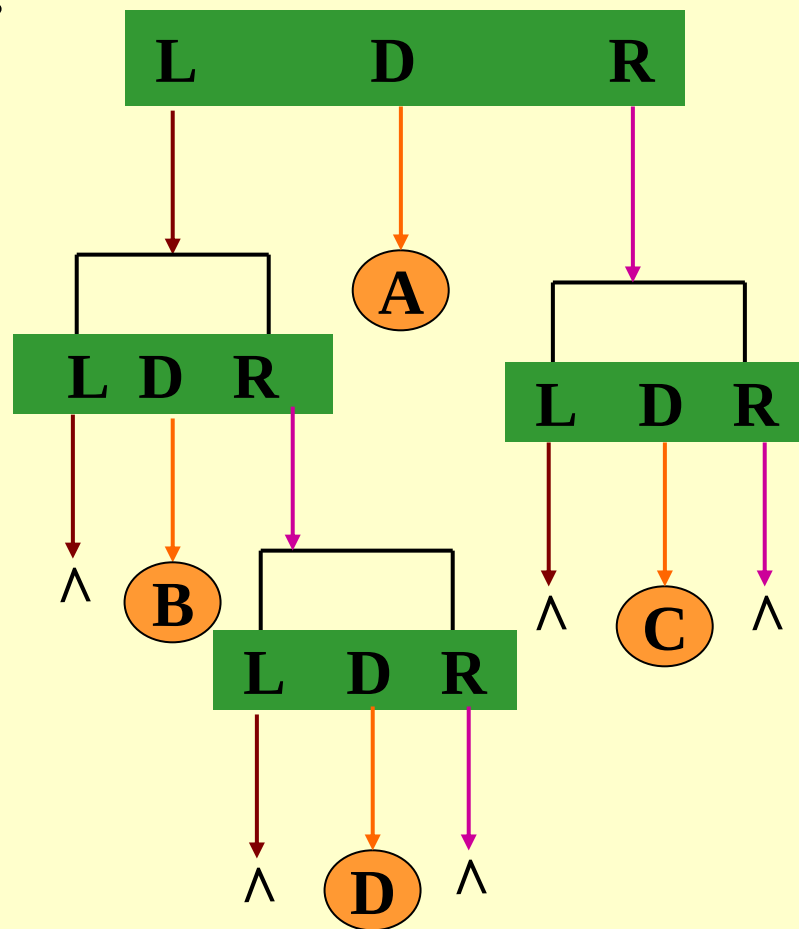
in-order中序遍历二叉树：若二叉树为空，则空操作；否则对每个结点：

- (1) 中序遍历左子树；
- (2) 访问根结点；
- (3) 中序遍历右子树。



The in-order traversal is :

B D A C

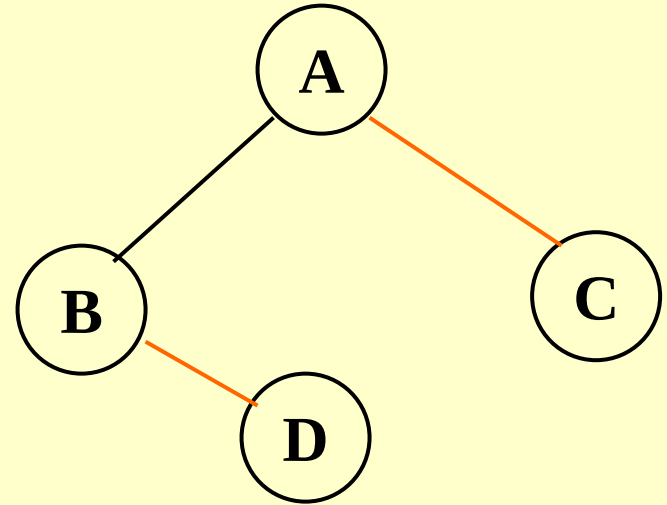


pre-order traversal

前序遍历、先序遍历

For each node, to

- (1) 访问根结点;
- (2) 先序遍历左子树;
- (3) 先序遍历右子树。



Pre-order traversal is:

A B D C

post-order traversal

后序遍历

For each node, to

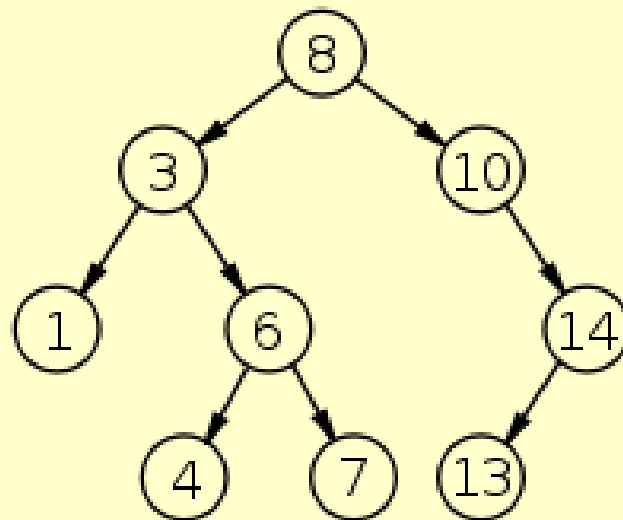
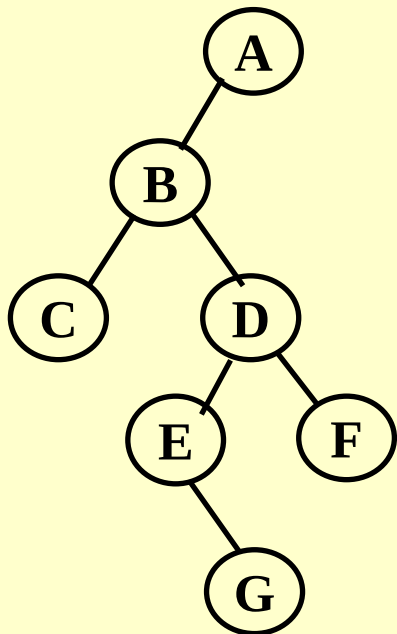
- (1) 后序遍历左子树;
- (2) 后序遍历右子树;
- (3) 访问根结点。

Post-order traversal is:

D B C A

👉 Tree Traversals —— 练习

给出三种遍历方式序列



❖ **Preorder Traversal**先序遍历

```
Status PreOrderRecursionTraverse(BiTree T)
{
    if(T){
        if(!Visit(T->data)) return ERROR;
        PreOrderRecursionTraverse(T->lchild);
        PreOrderRecursionTraverse(T->rchild);
    }
    return OK;
}
```

/*方法一：遍历过程中计数*/

int getNodesNumber(BiTree T)

{

static int i=0; //为什么用static类型

if(T){

i++;

getNodesNumber(T->lchild);

getNodesNumber(T->rchild);

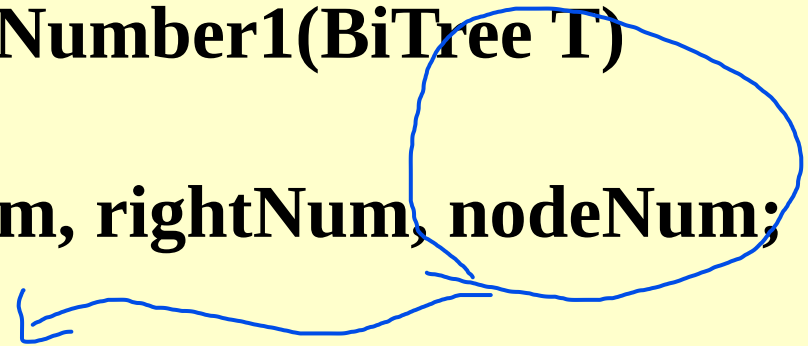
}

return i;

}

/*方法二：递归计数*/

```
int getNodesNumber1(BiTree T)  
{  
    int leftNum, rightNum, nodeNum;  
    if (!T)  
        treeNum = 0;  
    else {  
        leftNum = getNodesNumber1(_____);    // 左  
        rightNum = getNodesNumber1(_____);    // 右  
        nodeNum = _____ + _____ + 1;    // 中  
    }  
    return _____;  
}
```



/*方法三：递归计数优化*/

int getNodesNumber2(BiTree T)

{

if (T == NULL) return 0;

return 1 + getNodesNumber2(_____)

+ getNodesNumber2(_____);

}

Experiment 6-1 二叉树实现

Problems:

BiTree CreateBiTree(BiTree T);

Status PreOrderRecursionTraverse(BiTree T);

Status InOrderRecursionTraverse(BiTree T);

Status PostOrderRecursionTraverse(BiTree T);

int getNodesNumber(BiTree T);

int getLeafNumber(BiTree T);

int getHeight(BiTree T);

Status Visit(ElemType e);

§ 6.3 三种遍历之间的关系

力扣106.从中序与后序遍历序列构造二叉树

一、由前序和中序遍历，得到后序

- PreOrder: A B D G C E F H
- InOrder: D G B A E C H F
- PostOrder: ?

一、已知前序、中序遍历，求后序遍历

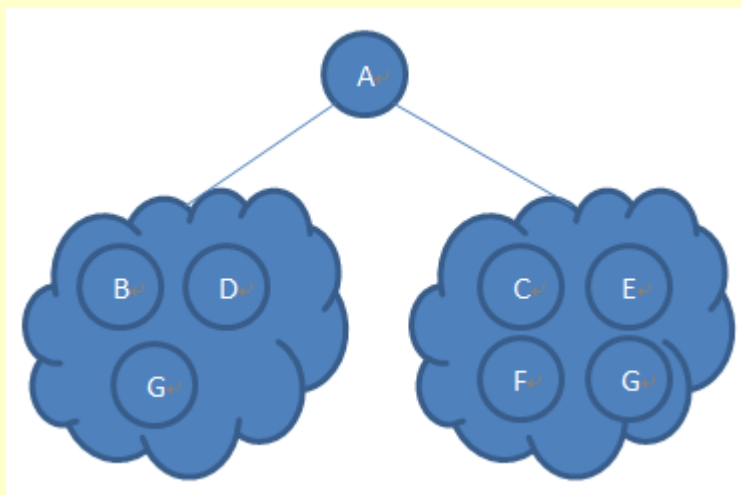
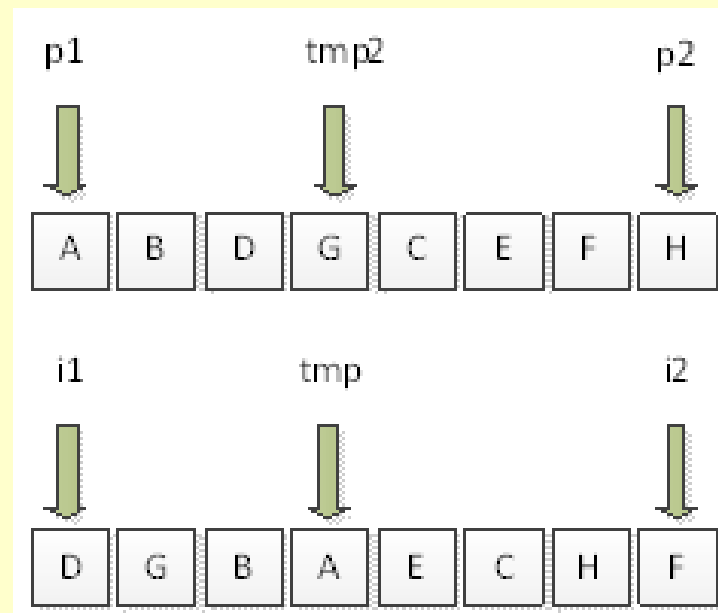
1. 根据前序遍历的特点，**根结点为A**；
2. 观察中序遍历，root节点A左侧的DGB必然是root的左子树，G右侧的AECHF必然是root的右子树。
3. 观察左子树DGB，左子树的中的根节点必然是大树的root的leftchild。在前序遍历中，大树的root的leftchild位于root之后，所以左子树的根节点为B。
4. root的右子树节点AECHF中的根节点也可以通过前序遍历求得。在前序遍历中，一定是先把root和root的所有左子树节点遍历完之后才会遍历右子树，并且遍历的左子树的第一个节点就是左子树的根节点。同理，遍历的右子树的第一个节点就是右子树的根节点。
5. 上面的过程是递归的。先找到当前树的根节点，然后划分为左子树，右子树，然后进入左子树重复上面的过程，然后进入右子树重复上面的过程。

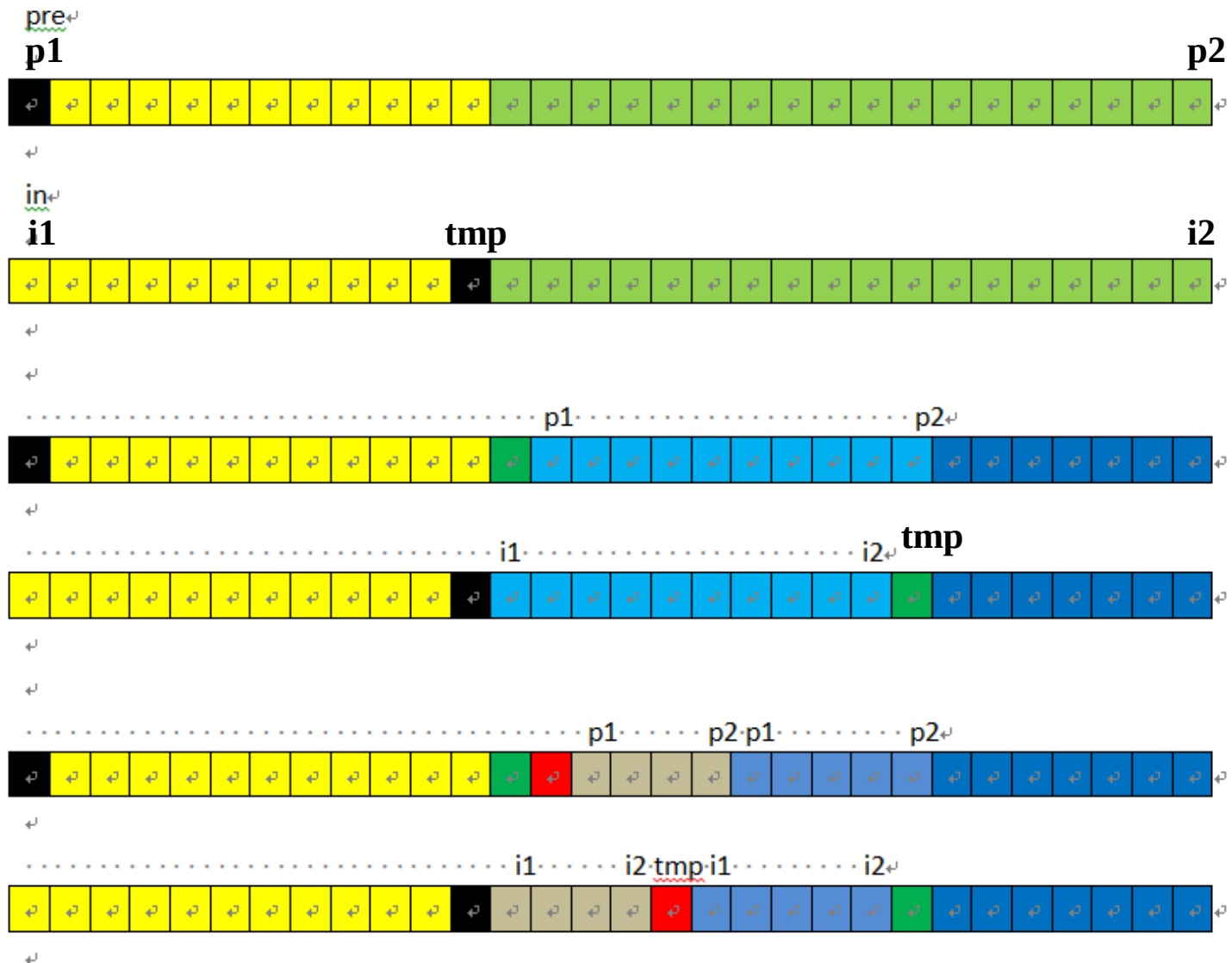
该步递归的过程如下：

- 1 确定根, 确定左子树, 确定右子树。
- 2 在左子树中递归。
- 3 在右子树中递归。
- 4 打印当前根。

PreOrder: A B D G C E F H

InOrder: D G B A E C H F





左子树 $pre: p1+1, p2$ 由 tmp 及 $i1$ 相对位置计算

$in: i1, tmp-1$

右子树 $pre: p1$ 由 tmp 及 $i1$ 相对位置计算, $p2$

$in: tmp+1, i2$

Experiment 6-2 由先序和中序遍历求后序

Problems:

- 给定PreOrder和InOrder, 返回 PostOrder
- 给定PostOrder和InOrder, 返回 PreOrder
- 能否由 PreOrder和PostOrder创建二叉树?

•Example:

InOrder: DBEGACF

PostOrder : DGEBFCA

Experiment 6-1b 二叉树实现（扩展）

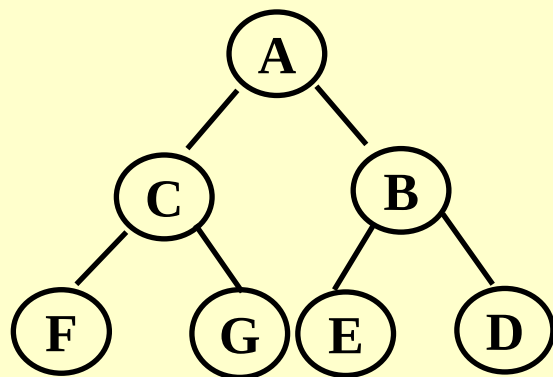
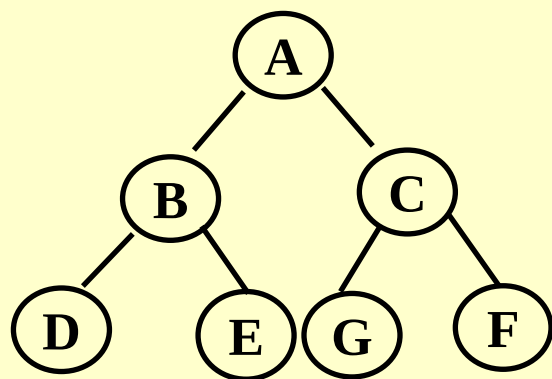
104. 二叉树的最大深度

226. 翻转二叉树

101. 对称二叉树

*/*递归*/*

```
BiTree invertTree(BiTree T){  
    BiTree temp;  
    if (____){ return ____; }//递归出口  
    else {  
        //交换结点的左右孩子  
        ____;  
        ____;  
        ____;  
        invertTree(____);  
        invertTree(____);  
    }  
    return T;  
}
```



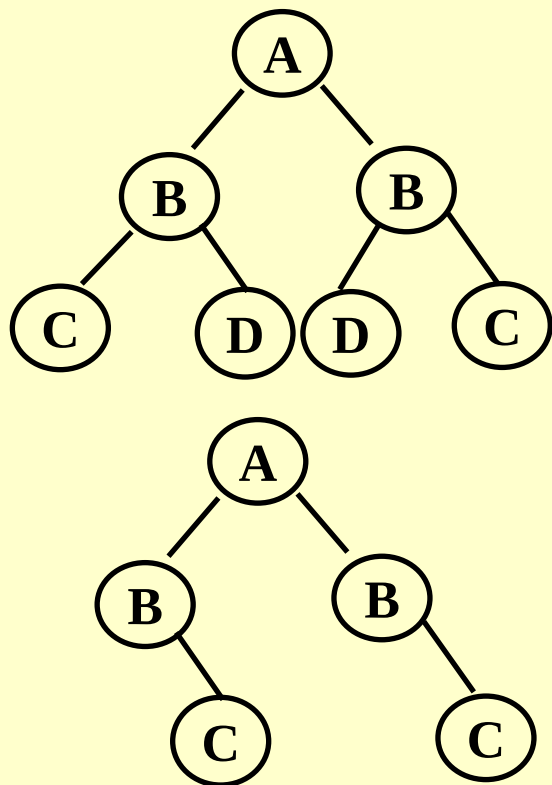
迭代法（前序遍历）
广度优先遍历(层次遍历)

Experiment 6-1b 二叉树实现（扩展）

104. 二叉树的最大深度

226. 翻转二叉树

101. 对称二叉树



100. 相同的树

572. 另一个树的子树

```
bool isSymmetric(BiTree T) {
    if (T == NULL) return TRUE;
    return compare(T->lchild, T->rchild);
}
```

```
bool compare(BiTree left, BiTree right) {
    bool outside, inside, isSame;
    // 首先排除空节点的情况
    if (____ && ____ ) return FALSE;
    else if (____ && ____ ) return FALSE;
    else if (____ && ____ ) return TRUE;
    // 排除了空节点，再排除数值不相同的情况
    else if (left->data != right->data) return FALSE;
    // 左右节点都不为空，且数值相同，递归下一层
    outside = compare(left->lchild, right->rchild);
    // 外侧节点：左子树左孩子，右子树右孩子
    inside = compare(____, ____);
    // 内侧节点：左子树右孩子，右子树左孩子
    isSame = outside && inside;
    return isSame;
}
```

如何优化？

1、假定在一棵二叉树中，双分支结点数为15，单分支结点数为30个，则叶子结点数为____个。

A、15 ~~B、16~~ C、17 D、47

2、按照二叉树的定义，具有3个结点的不同形状的二叉树有____种。(易)

A、3 B、4 ~~C、5~~ D、6

3、按照二叉树的定义，具有3个不同数据结点的不同的二叉树有____种。(易)

A、5 B、6 ~~C、30~~ D、32

4、深度为5的二叉树至多有____个结点。(易)

A、16 B、32 ~~C、31~~ D、10

5、设高度为h的二叉树上只有度为0和度为2的结点，则此类二叉树中所包含的结点数至少为____。

A、2h ~~B、2h-1~~ C、2h+1 D、h+1

6、对一个满二叉树，m个树叶，n个结点，深度为h，则____。(易)

A、 $n=h+m$ B、 $h+m=2n$ C、 $m=h-1$ ~~D、 $n=2^{h-1}$~~

7、任何一棵二叉树的叶结点在先序、中序和后序遍历序列中的相对次序____。(易)

~~A、不发生改变~~ B、发生改变 C、不能确定 D、以上都不对

8、某二叉树的前序遍历结点访问顺序是abdgcefh，中序遍历的结点访问顺序是dgbacfh，则其后序遍历的结点访问顺序是____。

A、bdgcefha B、gdbecfha C、bdgaechf ~~D、gdbefhca~~

9、在一非空二叉树的中序遍历序列中，根结点的右边____。(易)

~~A、只有右子树上的所有结点~~ B、只有右子树上的部分结点

C、只有左子树上的部分结点 D、只有左子树上的所有结点

10. 【2012统考真题】若一棵二叉树的前序遍历序列为a,e,b,d,c,后序遍历序列为b,c,d,e,a，则根结点的孩子结点()。

~~A、只有e~~ B、有e,b C、有e,c D、无法确定